

RI204
Windows programiranje
dr. sci. Edin Pjanić

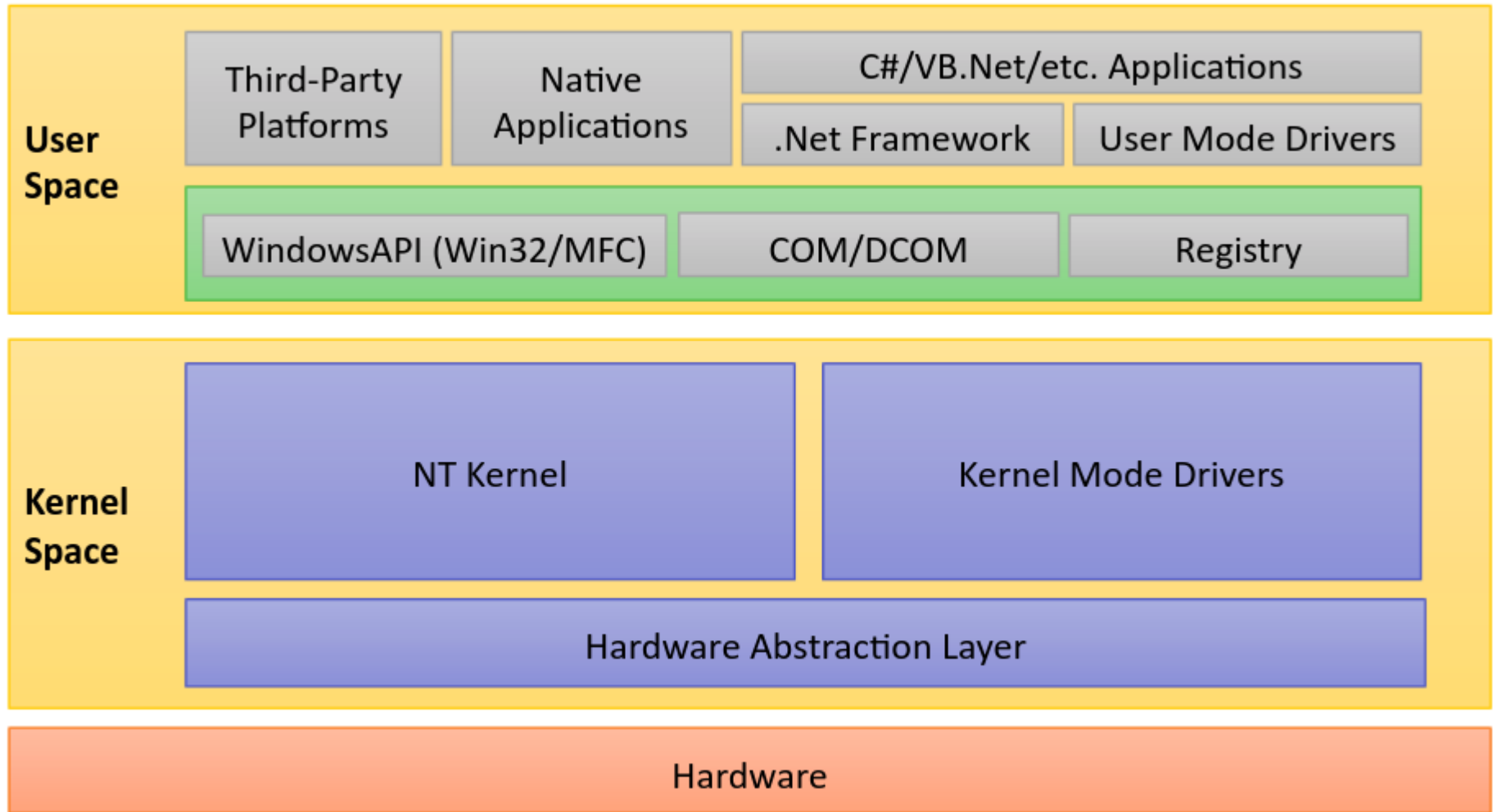
- Windows arhitektura
- Kreiranje jednostavnog programa
 - window klasa
 - window procedura
 - petlja windows programa (message loop)

- Osobine
 - Portabilan
 - Vrlo mali dio koda pisan u assembleru
 - Siguran
 - Svaki program dobija separadni memorijski prostor tzv. *user mode*
 - Kernel i device driveri izvršavaju se u privilegovanom *kernel* modu
 - Od 1995. čisti 32 bitni sistem (Win32)
 - 4 GB linearni adresni prostor
 - pre-emptive multitasking i multithreading
 - Operativni sistem odlučuje o vremenu koji će određena aplikacija da dobije za izvršenje na procesoru

- Windows API
 - Windows *application programming interface* (API) je skup funkcija koje omogućavaju komunikaciju sa Windows operativnim sistemom, npr:
 - Kreiranje prozora
 - Manipulacija memorijom
 - Kreriranje niti (*threads*) itd.
 - API funkcije nalaze se unutar dinamičkih biblioteka tzv. *dll*-ova (dynamic link libraries)
 - Funkcije se učitavaju dinamički pri izvršenju programa
 - Štedi se prostor na disku
 - Aplikacije se ne moraju rekompajlirati u slučaju promjene implementacije funkcija unutar *dll*-ova

Windows OS arhitektura

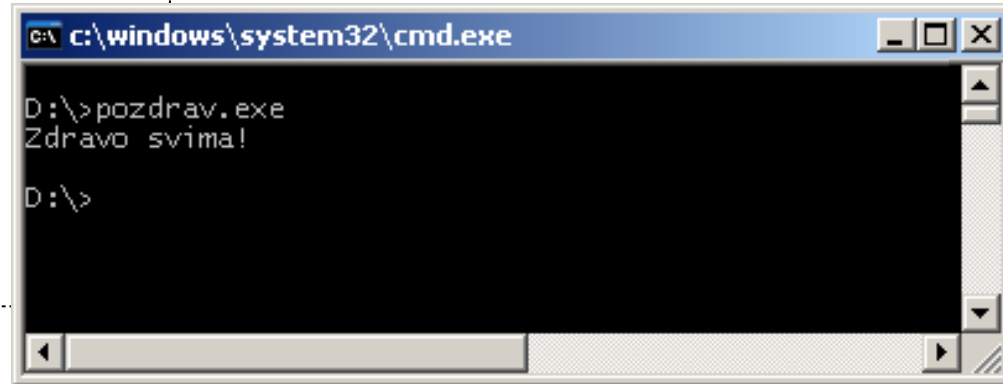
5



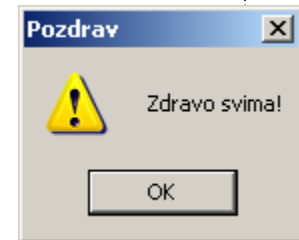
Komandni i vizuelni interfejs – “Hello, world!”

6

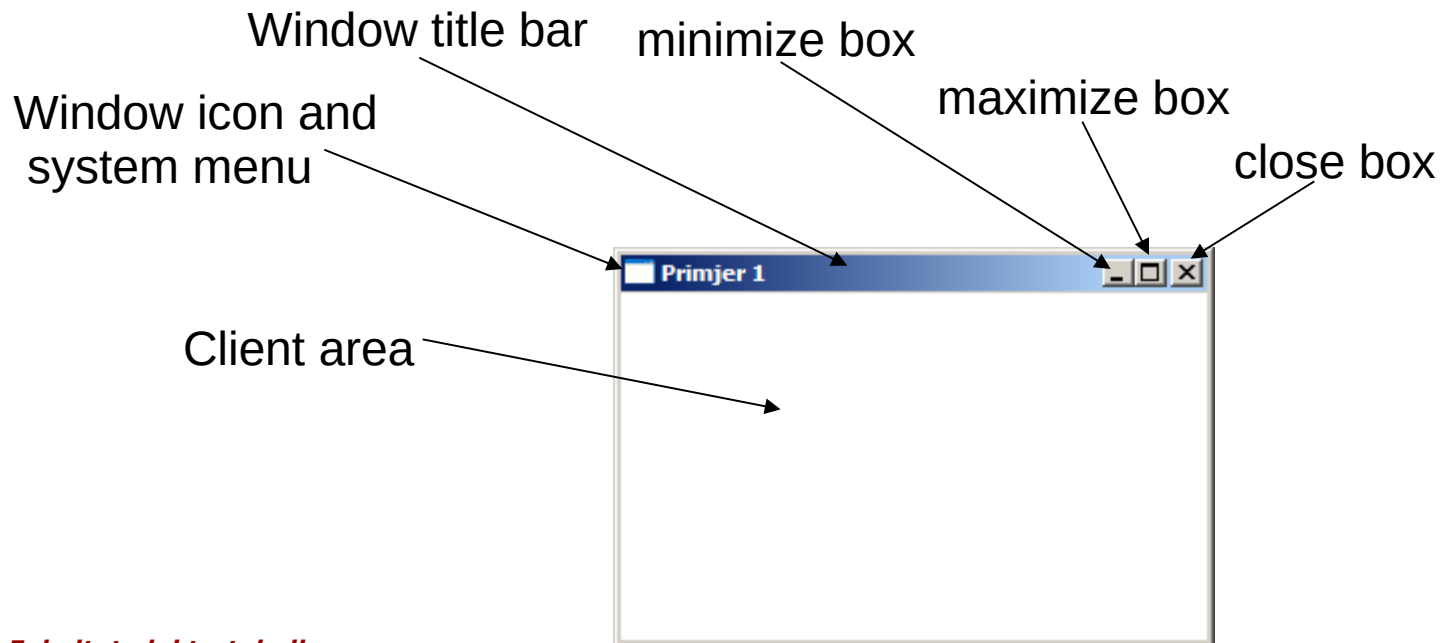
```
#include <stdio.h>
int main()
{
    printf("Zdravo svima!\n");
    return 0;
}
```



```
#include <windows.h>
int WINAPI WinMain(HINSTANCE hInstance,
                  HINSTANCE hPrevInstance,
                  LPSTR lpCmdLine,
                  int nCmdShow)
{
    MessageBox(NULL,
               "Zdravo svima!",
               "Pozdrav",
               MB_OK | MB_ICONWARNING);
    return 0;
}
```

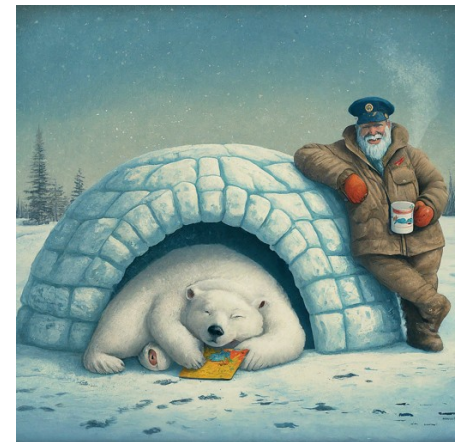
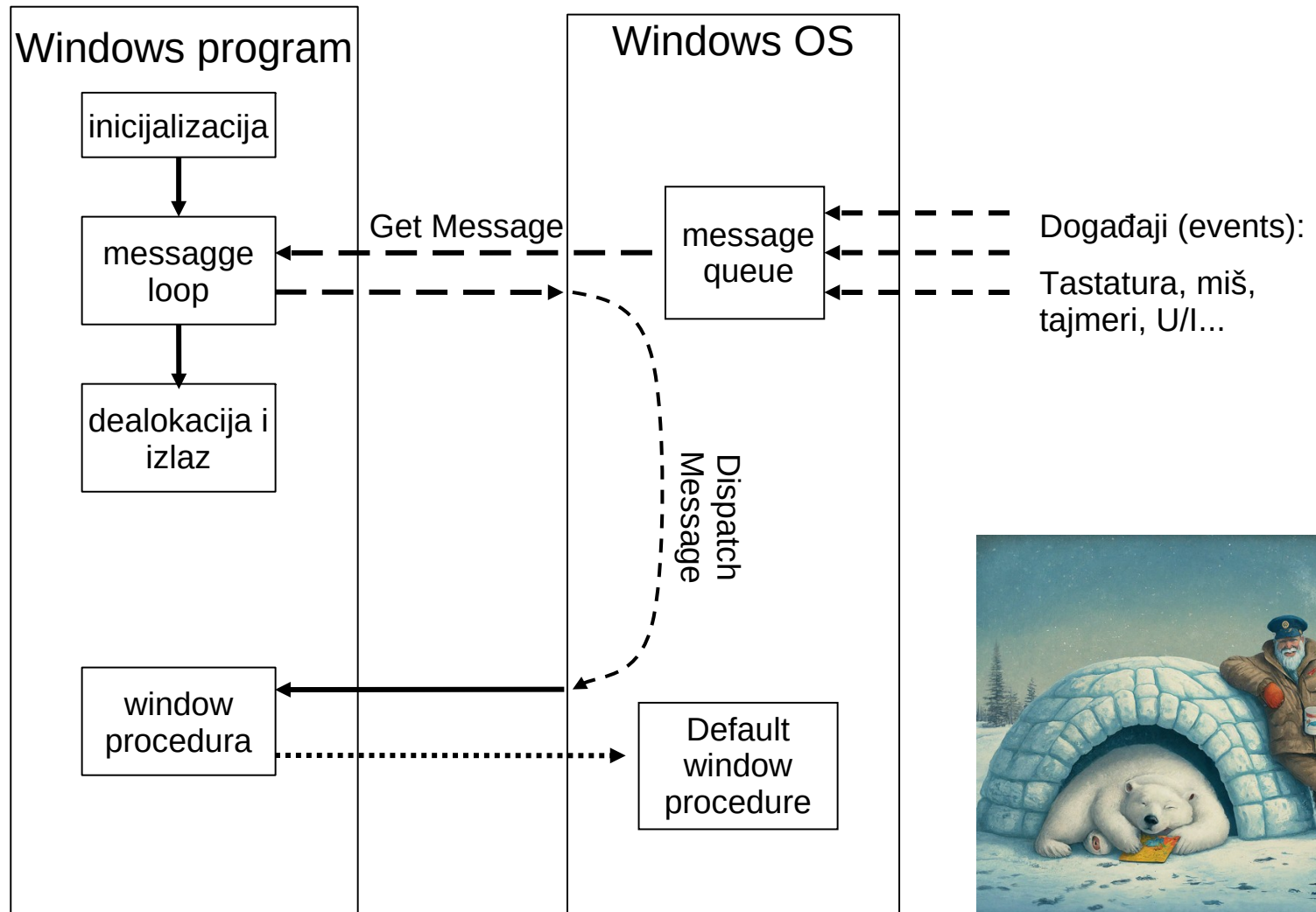


- Komunikacija sa korisnikom obavlja se preko prozora.
 - Prozor - pravougaona površina na ekranu koja prima korisnički ulaz (obično preko miša i tastature) i prikazuje izlaz u obliku teksta ili grafike.



- Program sa prozorima (*windows*) komunicira preko poruka (poruka
 - Poruka se može posmatrati kao notifikacija da se dogodio neki događaj (*event*) koji može a i ne mora zahtijevati neku akciju od strane programa.
 - Neki događaji generisani aktivnostima korisnika:
 - klik ili pomjeranje miša
 - promjena dimenzija prozora itd.
- Poruka je u osnovi lista parametara koji opisuju tip događaja.

- Program u pravilu ne tretira poruke direktno već se za svaki prozor definiše posebna funkcija
 - tzv. window (prozorska) procedura (*window procedure*)
- Window proceduru operativni sistem poziva kada se dogodi neki događaj koji je u vezi sa tim prozorom.



- Za kreiranje funkcionalnog windows programa sa prozorima potrebno je učiniti sljedeće korake:
 1. Kreirati window proceduru (definirati funkciju).
 2. Kreirati window klasu (varijabla odgovarajućeg tipa - strukture).
 3. Registrovati window klasu u OS-u.
 4. Na bazi registrovane window klase kreirati prozor.
 5. Kreirati petlju koja prima i dispečira poruke ka window proceduri.

- Za rad sa windows objektima najčešće se koristi tzv. handle
 - *Handle* je cijeli broj (32 bitni tipa DWORD - unsigned long) kojeg, kao identifikator, OS asocira sa raznim elementima, objektima i sl. Takvi objekti su interni za OS i nisu dostupni u adresnom prostoru windows aplikacije.
 - Aplikacija, koristeći *handle* i API funkcije, može posredno da čita i/ili mijenja svojstva windows objekata.
 - *Handle* se dodjeljuje za različite tipove objekata, kao što su:
 - instanca aplikacije,
 - sličica (ikonica),
 - meni (izbornik),
 - prozor itd.

- Klasični programi pisani za konzolu startaju se od funkcije `main`.
- Windows programi startaju se od funkcije `WinMain` koja uzima 4 parametra i ima sljedeći potpis:
 - `int WINAPI WinMain( HINSTANCE hInstance, HINSTANCE hPrevInstance, PSTR szCmdLine, int iCmdShow)`
 - `hInstance` – handle koji je OS generisao za aplikaciju.
 - `hPrevInstance` – parametar koji se više ne koristi, uvijek `NULL`.
 - `szCmdLine` – null-terminirani string parametara proslijeđenih sa komandne linije, bez naziva programa.
 - `iCmdShow` – cijeli broj proslijeđen aplikaciji prilikom pokretanja; indicira kako se prozor inicijalno prikazuje.

- U tijelu `WinMain` funkcije prije kreiranja prozora potrebno je opisati osnovni izgled i ponašanje našeg prozora. Ovaj korak izvodi se na način:
 - Definirati window klasu
 - `WNDCLASS wc;`
 - Postaviti polja klase.
 - Registrovati kreiranu klasu u OSu pozivanjem funkcije:
 - `RegisterClass(&wc);`
- Ovako definisana klasa može se koristiti za generisanje proizvoljnog broja prozora sa sličnim svojstvima.

```
typedef struct
```

```
{
```

```
    UINT        style; // stil klase
    WNDPROC      lpfnWndProc; // pointer na windows proceduru
    int          cbClsExtra; // ekstra prostor u strukturi windows klase
    int          cbWndExtra ; // ekstra prostor u strukturi prozora
    HINSTANCE     hInstance ; // handle za instancu aplikacije koja posjeduje win.proc.
    HICON         hIcon ; // handle za sličicu u uglu prozora (ikonu)
    HCURSOR       hCursor ; // handle za pointer misa
    HBRUSH        hbrBackground ; // handle za cetkicu kojom se farba prozor
    LPCTSTR       lpzMenuName ; // string koji sadrzi ime menu resursa
    LPCTSTR       lpzClassName ; // string koji sadrzi ime klase (max. dužine 256)
```

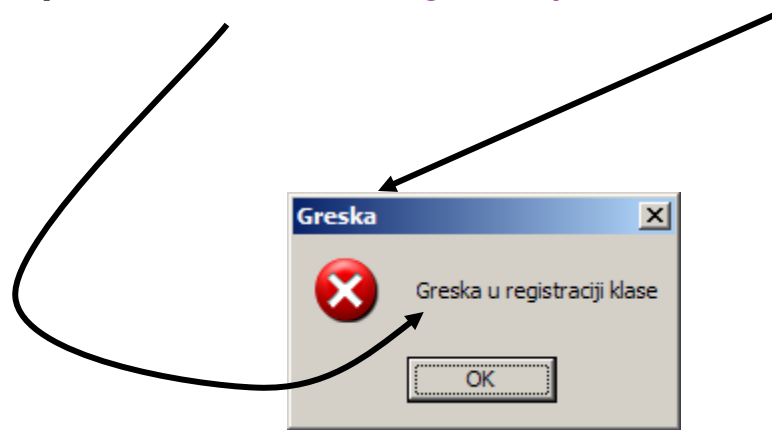
```
}
```

```
WNDCLASS;
```

```
wc.style           = CS_HREDRAW | CS_VREDRAW ;
wc.lpfnWndProc     = WndProc ;
wc.cbClsExtra      = 0 ;
wc.cbWndExtra      = 0 ;
wc.hInstance       = hInstance ;
wc.hIcon           = LoadIcon (NULL, IDI_APPLICATION) ;
wc.hCursor         = LoadCursor (NULL, IDC_ARROW) ;
wc.hbrBackground   = (HBRUSH) GetStockObject (WHITE_BRUSH) ;
wc.lpszMenuName    = NULL ;
wc.lpszClassName   = "Moja klasa";
```

- LoadIcon, LoadCursor i GetStockObject su API funkcije koje vraćaju *handle* na učitane resurse za ikonu, pointer za miša i četkice za bojenje.


```
if (!RegisterClass (&wc))  
{  
    MessageBox (NULL, "Greska u registraciji klase", "Greska", MB_ICONERROR) ;  
    return 0 ;  
}
```



- MessageBox je API funkcija kojom se u zasebnom prozoru vrši:
 - obavještavanje korisnika o eventualnoj grešci,
 - postavljanje nekog pitanja korisniku i sl.

```
HWND hwnd;  
hwnd = CreateWindow ("Moja klasa", // ime windows klase  
    "Primjer 1", // tekst u Window title bar-u  
    WS_OVERLAPPEDWINDOW, // stil prozora  
    100, // inicijalna x koordinata  
    100, // inicijalna y koordinata  
    300, // inicijalna sirina prozora  
    200, // inicijalna visina prozora  
    NULL, // handle na vlasnika (parent) prozora  
    NULL, // window menu handle  
    hInstance, // handle na instancu aplikacije  
    NULL) ; // dodatni kreacioni parametri  
ShowWindow (hwnd, iCmdShow) ;  
UpdateWindow (hwnd) ;
```

- CreateWindow kreira prozor i vraća handle tipa HWND na novokreirani objekat
- ShowWindow prikazuje prozor
- UpdateWindow šalje poruku prozoru da izvrši crtanje klijentskog područja

```
MSG  msg ;  
while (GetMessage (&msg, NULL, 0, 0) > 0)  
{  
    TranslateMessage (&msg) ;  
    DispatchMessage (&msg) ;  
}
```

- GetMessage se koristi za primanje poruka od OS-a
 - Poziv funkcije je smješten u petlju jer želimo da kontinuirano primamo poruke od OS-a.
 - Ako nema nove poruke, program “blokira”, tj. čeka na poruku.
 - Program napušta petlju kada funkcija vrati vrijednost FALSE.
 - Poruka se prima u lokalnu strukturu tipa MSG.
 - Drugi parametar je handle prozora od kojeg želimo da primamo poruke; za poruke od svih prozora aplikacije možemo postaviti NULL.
- TranslateMessage – vezana za procesiranje inputa sa tastature.
- DispatchMessage – prosljeđuje poruku OS-u koji nakon procesiranja “šalje” poruku u windows procedu

- Funkcija koja tretira poruke može biti bilo kojeg imena ali mora imati sljedeći potpis:

```
LRESULT CALLBACK WindowProc(  
    HWND hwnd, // handle prozora koji prima poruku  
    UINT msg,  // id poruke  
    WPARAM wparam, // dodatno definiše parametre poruke  
    LPARAM lparam // dodatno definiše parametre poruke  
);
```

- Drugi parametar definiše tip poruke, npr. WM_PAINT.
- U zavisnosti od ovog parametra (vrste poruke) treći i četvrti parametar imaju različita značenja.

```
LRESULT CALLBACK WndProc (HWND hwnd, UINT message, WPARAM wParam, LPARAM lParam)
{
    switch (message)
    {
        case WM_DESTROY:
            PostQuitMessage (0) ;
            return 0 ;
    }
    return DefWindowProc (hwnd, message, wParam, lParam) ;
}
```

- Tip poruke obično se detektuje unutar switch izraza a zatim za specifičan tip poruke implementira određeni kod.
 - Neke poruke:
 - WM_CREATE – kreiranje prozora
 - WM_PAINT – crtanje klijentskog područja
 - WM_DESTROY – uništenje prozora
- DefWindowProc je defaultna window procedura koja procesira sve poruke koje korisnička procedura nije tre